

Attempt All Questions :**1. Creating a Point Class (Easy)**

Write a class Point that represents a point in 2D space with x and y coordinates. Implement the following:

- A default constructor.
- A parameterized constructor to initialize x and y.
- A display function to print the point's coordinates.

2. Complex Number Operations (Moderate)

Create a class Complex that represents a complex number with real and imaginary parts. Implement:

- A default constructor and a parameterized constructor.
- A member function add to add two complex numbers.
- A member function subtract to subtract one complex number from another.
- A member function display to print the complex number in the form $a + bi$.

3. Student Information System (Moderate)

Design a class Student with the following members:

- name (string)
- rollNo (int)
- marks (array of 5 subject marks)

Provide member functions to:

- Input details of the student.
- Calculate the total and average marks.
- Display the student details and the calculated total and average marks.

4. Class Date Manipulation (Moderate)

Implement a class Date with attributes day, month, and year. The class should:

- Include constructors for default and parameterized initialization.
- A function isLeapYear() to determine if the year is a leap year.
- A function displayDate() to print the date in DD/MM/YYYY format.

5. Rectangle Class with Area Calculation (Easy)

Write a class Rectangle with attributes length and width. Include:

- A constructor to initialize the rectangle's dimensions.
- A member function area to calculate and return the area of the rectangle.
- A member function display to print the dimensions and area.

6. Bank Account Management (Moderate)

Create a class BankAccount with:

- Attributes: accountNumber, balance.
- A constructor to initialize the balance.

- Member functions to:
 - Deposit an amount.
 - Withdraw an amount.
 - Display the account details.
-

7. Class for Employee Payroll System (Moderate)

Create a class Employee with:

- Attributes: name, ID, and basicSalary.
 - A constructor for initializing these attributes.
 - Member functions to:
 - Calculate grossSalary as basic salary + 20% HRA + 10% DA.
 - Display employee details and gross salary.
-

8. Class for Circle Geometry (Easy)

Write a class Circle with a private data member radius. The class should include:

- A constructor to initialize the radius.
 - A member function area to calculate and return the area.
 - A member function circumference to calculate and return the circumference.
 - A display function to show the circle's radius, area, and circumference.
-

9. Temperature Conversion (Moderate)

Implement a class Temperature that contains:

- A data member celsius.
 - A constructor to initialize celsius.
 - Member functions to:
 - Convert Celsius to Fahrenheit.
 - Convert Celsius to Kelvin.
 - Display both conversions.
-

10. Class for Time Representation (Moderate)

Create a class Time with:

- Attributes: hours, minutes, and seconds.
 - A constructor to initialize these attributes.
 - A member function displayTime() to display the time in HH:MM:SS format.
 - A member function addTime() to add two Time objects and return the result.
-

11. Inventory Management System (Moderate)

Create a class Item with:

- Attributes: itemCode, itemName, and price.
 - A constructor for initialization.
 - A member function applyDiscount(double percent) that reduces the price by the given percentage.
 - A member function display to show item details.
-

12. Rational Numbers Operations (Moderate)

Write a class Rational for performing operations on rational numbers with attributes numerator and denominator. Include:

- A constructor for initialization.
- Member functions for adding, subtracting, multiplying, and dividing two rational numbers.
- A member function to display the rational number in fraction form.

13. Movie Ticket Booking System (Moderate)

Create a class MovieTicket that represents a ticket for a movie show. Include:

- Attributes: movieName, seatNumber, price.
- Constructors to initialize these attributes.
- A member function to print the ticket details.
- A member function to calculate a discount if the user books more than 3 tickets.

14. Currency Conversion (Easy)

Create a class CurrencyConverter that converts an amount from one currency to another. Include:

- A constructor to initialize the amount in a specific currency.
- Member functions to convert the amount to other currencies like USD, EUR, and GBP.
- A member function to display the converted amounts.

15. Book Management System (Moderate)

Write a class Book with attributes:

- title, author, price, and stock.
- A constructor for initialization.
- Member functions to display book details and to check if a book is in stock.

16. Library Management System with Book and Member Classes (Complex)

Design a system for a library with the following requirements:

- A class Book with attributes title, author, isbn, quantity, and price.
- A class Member with attributes name, memberID, issuedBooks (array of issued book ISBNs), and maxBooks (limit on the number of books a member can issue).
- A class Library that manages an array of Book objects and Member objects, allowing:
 - Issuing a book to a member (check stock and maxBooks limit).
 - Returning a book.
 - Displaying the current list of issued books for a member.

17. Student Grading System (Complex)

Implement a base class Person with basic details such as name and age. Derive two classes:

- Student with additional attributes rollNo and marks (array of subject marks).
- Teacher with attributes teacherID and salary.

Write a function `calculateGrade()` in the `Student` class that calculates the grade based on marks and displays the student's details. Use the concept of inheritance to manage common attributes in `Person` and demonstrate polymorphism by overriding a `virtual display()` function in both classes.

18. Ticket Booking System Using Multiple Classes (Complex)

Create a ticket booking system with the following classes:

- Event class to store details about an event (eventName, date, venue, totalSeats).
 - Ticket class to store ticket information (ticketID, seatNumber, price).
 - Booking class to manage the ticket booking process. The Booking class should:
 - Book tickets for an event (checking seat availability).
 - Allow cancellation of a ticket (update available seats).
 - Display all booked tickets for an event. Demonstrate the use of multiple classes working together and implement proper object interactions between them.
-

19. Banking System with Transaction History (Complex)

Design a banking system with the following:

- A class `BankAccount` with attributes `accountNumber`, `accountHolder`, and `balance`.
 - A class `Transaction` with details of each transaction: `transactionID`, `amount`, `type` (deposit/withdrawal), and `date`.
 - A class `Bank` that manages an array of `BankAccount` and `Transaction` objects, providing the ability to:
 - Deposit or withdraw funds from an account.
 - Maintain a transaction history for each account.
 - Display account details along with the last 5 transactions.
-

20. Hospital Management System (Complex)

Create a hospital management system using the following:

- A class `Person` with common attributes like `name`, `age`, and `gender`.
- A derived class `Patient` with additional details such as `patientID` and `disease`.
- A derived class `Doctor` with details like `doctorID` and `specialization`.
- A class `Hospital` that maintains arrays of `Patient` and `Doctor` objects. The hospital should:
 - Assign a doctor to a patient based on specialization.
 - Display the list of patients under a specific doctor.
 - Provide a summary report of the hospital's total number of patients and doctors.